

PROVISIONAL PATENT APPLICATION

**METHOD AND SYSTEM FOR VERIFIABLE DELAY FUNCTION-
PROTECTED
BATCH RANDOM NUMBER GENERATION
WITH ANTI-PREVIEW GUARANTEES**

可驗證延遲函數保護之批次隨機數生成方法與系統
——具反預覽保證

Inventor: [Inventor Name]

Date of Filing: [Filing Date]

Prepared for: Saint Island International Patent & Law Offices

聖島國際專利商標聯合事務所

ATTORNEY-CLIENT PRIVILEGED & CONFIDENTIAL

CROSS-REFERENCE TO RELATED APPLICATIONS

[0001] This application is related to U.S. Patent Application Serial No. [Patent #1 Serial Number] (hereinafter "Patent #1"), entitled "Method and System for Gaming Random Number Generation," filed [date], which describes a Merkle-tree-based random number generation and verification system with blockchain anchoring.

[0002] The present invention extends Patent #1 by addressing a specific vulnerability in batch random number generation systems: the **insider preview attack**. Where Patent #1 provides post-commitment integrity (proving that random values were not changed after the Merkle root was committed), the present invention provides pre-commitment unpredictability (proving that random values could not have been known before the commitment was made).

Patent #1: Merkle proves values were not changed after commitment.

Patent #2 (this application): VDF proves values were not known before commitment.

[0003] Together, Patent #1 and the present invention provide a complete temporal integrity guarantee: the random values were neither predictable before generation nor alterable after generation. This two-sided guarantee — anti-preview plus anti-tampering — is the foundation of the Trust Source layer in the Computational Trust Infrastructure.

FIELD OF THE INVENTION

[0004] The present invention relates to systems and methods for generating cryptographically secured random numbers in batch using Verifiable Delay Functions (VDFs) to provide anti-preview guarantees. More specifically, the invention relates to a pipeline that

integrates VDF computation, distributed randomness beacons, deterministic batch generation, Merkle tree construction, blockchain anchoring, and pool-based token serving — ensuring that neither the system operator nor any party can preview, predict, or pre-compute the random values before the batch generation is complete.

BACKGROUND OF THE INVENTION

The Insider Preview Vulnerability

[0005] In systems that pre-generate batches of random numbers for later consumption — a common architecture in high-throughput gaming, AI agent systems, and enterprise applications — a fundamental vulnerability exists that is not addressed by Merkle-tree-based verification alone.

[0006] A Merkle tree provides post-commitment integrity: once the root hash is computed and anchored (e.g., to a blockchain), any modification to any leaf value in the tree is cryptographically detectable. This guarantees that the random numbers were not tampered with after the batch was generated. However, Merkle trees provide no guarantee about what happened *before* the root was committed.

[0007] Specifically, in a pure Merkle-based batch system, the following attack is possible:

- a. The system operator generates a batch of N random values and constructs a Merkle tree.
- b. Before anchoring the root hash to the blockchain, the operator previews the values.

- c. The operator selects which batch to anchor — choosing a batch whose values are favorable to the operator (or unfavorable to specific consumers).
- d. The anchored batch passes all Merkle verification checks because the values were never modified after commitment.

[0008] This is the **insider preview attack**: the operator does not need to modify the values (which Merkle verification would detect) — the operator only needs to *see* them before committing, and then selectively commit favorable batches. The Merkle tree is cryptographically intact, but the process was manipulated.

[0009] No existing batch random number generation system adequately addresses this vulnerability. Specifically:

[0010] Hardware Random Number Generators (HRNGs): HRNGs produce entropy from physical processes but provide no mechanism to prove that the operator did not preview or filter the output before batch construction.

[0011] Pseudo-Random Number Generators (PRNGs): PRNGs are deterministic given a seed. If the seed is known before batch generation, the entire batch is predictable. The insider preview attack is trivially exploitable.

[0012] Verifiable Random Functions (VRFs): VRFs (e.g., Chainlink VRF, as described in US12401532B2) prove that a specific output was derived from a specific input using a specific key. However, VRFs are typically designed for single-value generation, not batch generation. VRFs also rely on a secret key held by the generator — the holder of the secret key can compute the VRF output for any input, enabling preview of future outputs given future inputs. Furthermore, VRFs do not inherently introduce a time-lock: the VRF computation is fast, allowing rapid generation and evaluation of multiple candidate outputs.

[0013] Commit-Reveal Schemes: Traditional commit-reveal requires interaction between parties, introducing latency, coordination overhead, and a requirement for all parties to be online simultaneously. Commit-reveal does not scale to batch generation of thousands of values for pool-based serving.

[0014] There exists a need for a system that provides anti-preview guarantees for batch random number generation — proving that the values could not have been known, selected, or filtered before the batch was committed — while maintaining the sub-millisecond serving latency required for production applications.

Verifiable Delay Functions

[0015] A Verifiable Delay Function (VDF) is a cryptographic primitive that takes a prescribed amount of sequential computation time T to evaluate, even on a parallel computer, and produces an output that can be efficiently verified (typically in $O(\log T)$ time). VDFs were formally defined by Boneh et al. (2018, IACR ePrint 2018/601) and have been proposed for various applications including randomness beacons and consensus protocols.

[0016] The key properties of VDFs relevant to the present invention are:

- a. **Sequential computation:** The output cannot be computed faster than time T , regardless of the number of parallel processors available. This property is what provides the anti-preview guarantee.
- b. **Efficient verification:** Given the input, output, and proof, anyone can verify the computation in time much less than T (typically $O(\log T)$). This enables efficient audit.
- c. **Determinism:** Given the same input, the VDF always produces the same output. This enables reproducibility and independent verification.

[0017] While VDFs have been proposed and implemented in various contexts (Tezos consensus, StarkWare VeeDo, Ethereum research), no prior art describes the integration of VDFs into a batch random number generation pipeline with the specific combination of features described in the present invention: VDF-seeded deterministic batch generation, Merkle tree construction, blockchain anchoring, pool-based serving, distributed beacon integration, and multi-layer fallback architecture.

SUMMARY OF THE INVENTION

[0018] The present invention provides a method and system for generating batches of random numbers with verifiable anti-preview guarantees, comprising:

1. **A VDF-protected seed generation pipeline** that ensures the batch seed cannot be known before a minimum sequential computation time T has elapsed, preventing the system operator or any party from previewing the batch values before they exist.
2. **A distributed randomness beacon integration** (e.g., drand) that provides publicly verifiable entropy as the VDF input seed, ensuring the seed itself is unpredictable and verifiable by any third party.
3. **A hybrid entropy architecture** with automatic fallback from distributed beacon to local VDF computation, maintaining anti-preview guarantees even when the external beacon is unavailable.
4. **A deterministic batch derivation function** that transforms the VDF output into N random values organized in a Merkle tree, where N may be up to 65,536 or more.
5. **An asynchronous anchoring architecture** that decouples token availability from blockchain confirmation, enabling immediate pool injection while anchoring proceeds

in the background.

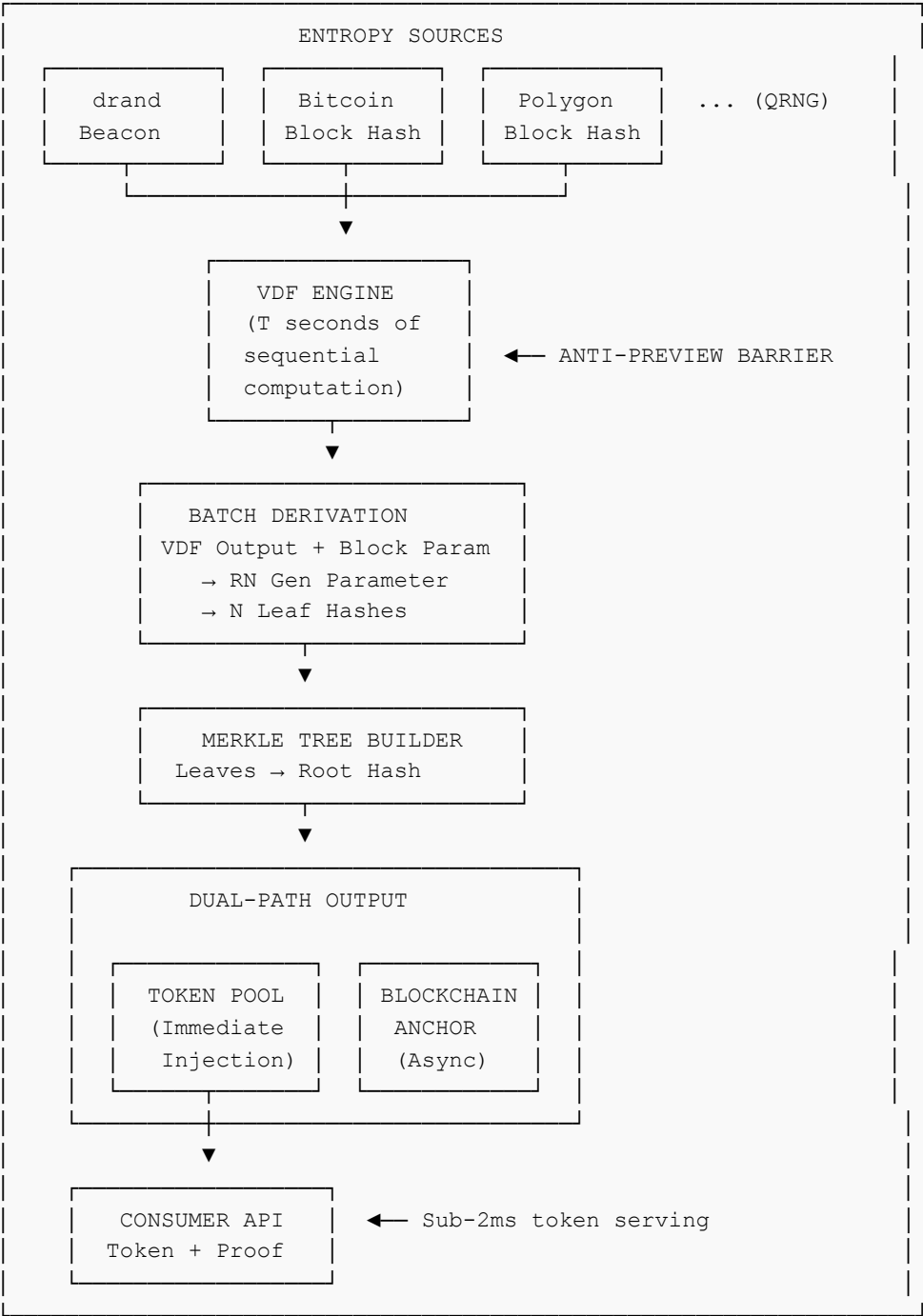
6. **A multi-worker VDF computation system** that enables parallel generation of multiple batches while maintaining the sequential (anti-preview) property within each individual batch.
7. **A shared pool architecture** for high-throughput serving at sub-2ms latency from pre-generated, VDF-protected token pools.

DETAILED DESCRIPTION OF PREFERRED EMBODIMENTS

System Architecture Overview

[0019] FIG. 1 illustrates the overall system architecture of the VDF-protected batch random number generation system. The system comprises the following major components:

FIG. 1 — System Architecture Overview



VDF-Protected Seed Generation

[0020] The core innovation of the present invention is the VDF-protected seed generation pipeline. The pipeline operates as follows:

[0021] Step 1 — Entropy Acquisition. The system obtains an initial entropy value from one or more independent sources. In the preferred embodiment, the primary entropy source is a distributed randomness beacon (drand), which produces publicly verifiable random values at regular intervals (approximately every 3 seconds). The drand beacon's output is verified against the beacon's published public key using BLS signature verification, specifically: the system confirms that $\text{SHA-256}(\text{signature})$ equals the randomness value. If the drand beacon is unavailable (network failure, timeout, consecutive failures exceeding a configurable threshold), the system falls back to a local entropy source combining operating system cryptographic randomness with blockchain block hash data.

[0022] Step 2 — Seed Construction. The entropy value is combined with a block parameter derived from a proof-of-work or proof-of-stake blockchain (e.g., Bitcoin block hash, Polygon block hash) to produce a VDF input seed. The combination uses a cryptographic hash function (SHA-256): $\text{vdfSeed} = \text{SHA-256}(\text{blockParam} + \text{randomEntropy})$. This ensures that the VDF seed depends on multiple independent sources of entropy, none of which is individually controllable by the system operator.

[0023] Step 3 — VDF Computation. The VDF input seed is processed through a Verifiable Delay Function requiring a minimum sequential computation time of T seconds (configurable; default $T=4$ seconds in the preferred embodiment). The VDF implementation uses iterative cryptographic hashing with a sequential dependency chain: each iteration's input depends on the previous iteration's output, preventing parallelization. In the preferred embodiment, the VDF performs $T \times 10,000$ sequential SHA-256 hash operations, where each hash takes as input the

concatenation of the previous hash output and the iteration counter. The VDF produces two outputs: the VDF output value and a VDF proof that enables efficient verification.

[0024] Anti-Preview Property. The VDF computation time T creates a temporal barrier: no party — including the system operator — can know the VDF output (and therefore cannot know any of the random values derived from it) until T seconds of sequential computation have elapsed. This is the anti-preview guarantee. Even if the operator knows the VDF input seed immediately, the output cannot be computed faster than T seconds. During this T -second window, the operator cannot preview, select, or filter the batch values. This fundamentally differs from all non-VDF batch generation systems, where the operator can compute the batch values instantaneously and preview them before commitment.

[0025] Step 4 — RN Generation Parameter. The VDF output is combined with the block parameter to produce the RN (Random Number) Generation Parameter: $\text{rnGenParam} = \text{SHA-256}(\text{vdfOutput} + \text{blockParam})$. This parameter serves as the deterministic seed for the entire batch. Given the rnGenParam , the batch is fully determined — but the rnGenParam itself was unknowable until the VDF computation completed.

Deterministic Batch Derivation

[0026] From the RN Generation Parameter, the system deterministically generates N random values (leaf hashes) as follows:

- a. For each leaf index i ($0 \leq i < N$), a unique node identifier is constructed: $\text{nodeId} = \text{"node-"} + \text{zeroPad}(i, 6) + \text{"-"} + \text{batchId}[6:14]$
- b. Each leaf hash is computed as: $\text{leafHash} = \text{SHA-256}(\text{rnGenParam} + \text{nodeId})$

- c. Each leaf hash is converted to a numeric random value by extracting the first 8 hexadecimal characters and dividing by 0xFFFFFFFF, producing a uniformly distributed value in $[0, 1)$.

[0027] The batch size N is configurable. In the preferred embodiment, $N = 65,536$ (2^{16}), which balances generation time, memory usage, and pool depth. The derivation is fully deterministic: given the same `rnGenParam` and batch metadata, any party can independently regenerate the identical batch, enabling complete audit and verification.

Merkle Tree Construction and Anchoring

[0028] The N leaf hashes are organized into a binary Merkle tree using pairwise SHA-256 hashing. The Merkle root hash represents a cryptographic commitment to all N values in the batch. This root hash is anchored to a blockchain (Polygon, in the preferred embodiment) via a smart contract transaction, creating an immutable, timestamped commitment.

[0029] Each individual token can be verified against the Merkle root using a proof path of $O(\log N)$ hashes — for $N=65,536$, this is 16 hash comparisons. The verification can be performed by any party with access to the token value, proof path, and on-chain root hash — no cooperation from the system operator is required.

Asynchronous Anchoring Architecture

[0030] A key architectural innovation of the present invention is the decoupling of token availability from blockchain anchoring. In the preferred embodiment:

- a. Tokens are injected into the serving pool immediately upon batch generation (before blockchain anchoring).

- b. Blockchain anchoring proceeds asynchronously in the background, with configurable retry logic (default: 3 attempts with exponential backoff).
- c. Tokens served before anchoring is confirmed are flagged with a "pending" anchor status. They carry full Merkle proofs and VDF provenance but cannot yet be verified against the blockchain until the anchor transaction is confirmed.
- d. Upon anchor confirmation, the batch metadata is updated with the transaction hash and block number, and all tokens from that batch are retroactively upgradeable to "anchored" verification level.

[0031] This architecture enables sub-2ms token serving latency — tokens are served from an in-memory pool without waiting for blockchain confirmation (which may take seconds to minutes). The anti-preview guarantee is provided by the VDF computation, not by the blockchain anchor. The blockchain anchor provides temporal proof and immutability, complementing the VDF's anti-preview guarantee.

Multi-Worker VDF Architecture

[0032] The system employs multiple VDF worker threads (configurable; default: 3 workers) to enable parallel generation of multiple batches. Each worker processes one batch at a time. The sequential property of VDF is maintained within each batch — a single batch cannot be generated faster by adding workers. However, multiple batches can be generated concurrently by assigning them to different workers.

[0033] A priority-based refill queue manages worker allocation. Exhaustion-triggered refills (pool completely depleted) receive the highest priority (10). Threshold-triggered refills (pool below configurable threshold, default 42% for gaming verticals, 50% for shared pools) receive medium priority (8). Initial pre-warming receives lowest priority (1).

Shared Pool and Dedicated Pool Architecture

[0034] The system supports two pool modes:

- a. **Shared Pool:** A single pre-warmed pool serves all consumers in designated verticals (e.g., AI agents). The shared pool is pre-warmed at system startup with a configurable number of batches (default: 8 batches = 524,288 tokens). Consumers are individually rate-limited (configurable; default: 100 requests per minute). New consumers can begin consuming tokens immediately — no per-consumer batch generation delay. Refill is triggered at 50% depth (threshold refill) and 20% depth (critical multi-batch emergency refill).
- b. **Dedicated Pool:** Per-consumer pools for verticals requiring isolation (e.g., regulated gaming). Each consumer has an independent pool with independent refill triggers, batch history, and audit trail. Dedicated pools are pre-warmed with one batch upon registration.

Hybrid Entropy Source Architecture

[0035] The system implements a hierarchical entropy source strategy with automatic failover:

- a. **Primary: Distributed Randomness Beacon (drand).** The system queries the drand quicknet chain (chain hash: 52db9ba...) for the latest beacon round. The beacon's BLS signature is verified against the chain's public key. If verification succeeds, the beacon's randomness value is used as the VDF input entropy. drand is preferred because its randomness is publicly verifiable by anyone — not just the system operator.

b. **Secondary: Local VDF with OS Entropy.** If the drand beacon is unavailable (timeout, consecutive failures exceeding a configurable threshold, integrity check failure), the system falls back to local entropy: operating system cryptographic randomness (`crypto.randomBytes`) combined with blockchain block hash data, processed through the local VDF engine. The anti-preview guarantee is maintained because the VDF computation time is the same regardless of entropy source.

[0036] The entropy source used for each batch is recorded in the batch metadata, providing transparent provenance. Consumers and auditors can distinguish between drand-sourced batches (publicly verifiable entropy) and locally-sourced batches (operator-verifiable entropy). This transparency is a core property of the system — there are no silent fallbacks.

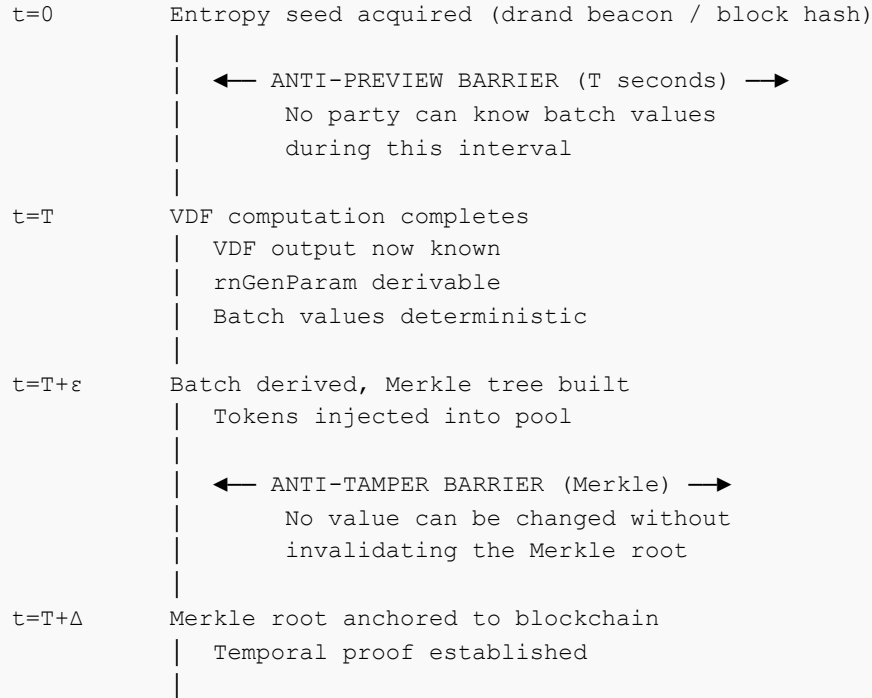
BRIEF DESCRIPTION OF THE DRAWINGS

FIG. 1 — System architecture overview showing the VDF-protected batch generation pipeline from entropy sources through VDF computation, batch derivation, Merkle tree construction, and dual-path output (pool injection and blockchain anchoring).

FIG. 2 — VDF anti-preview timeline showing the temporal barrier between seed acquisition and batch value availability.

FIG. 2 — VDF Anti-Preview Timeline

Timeline:

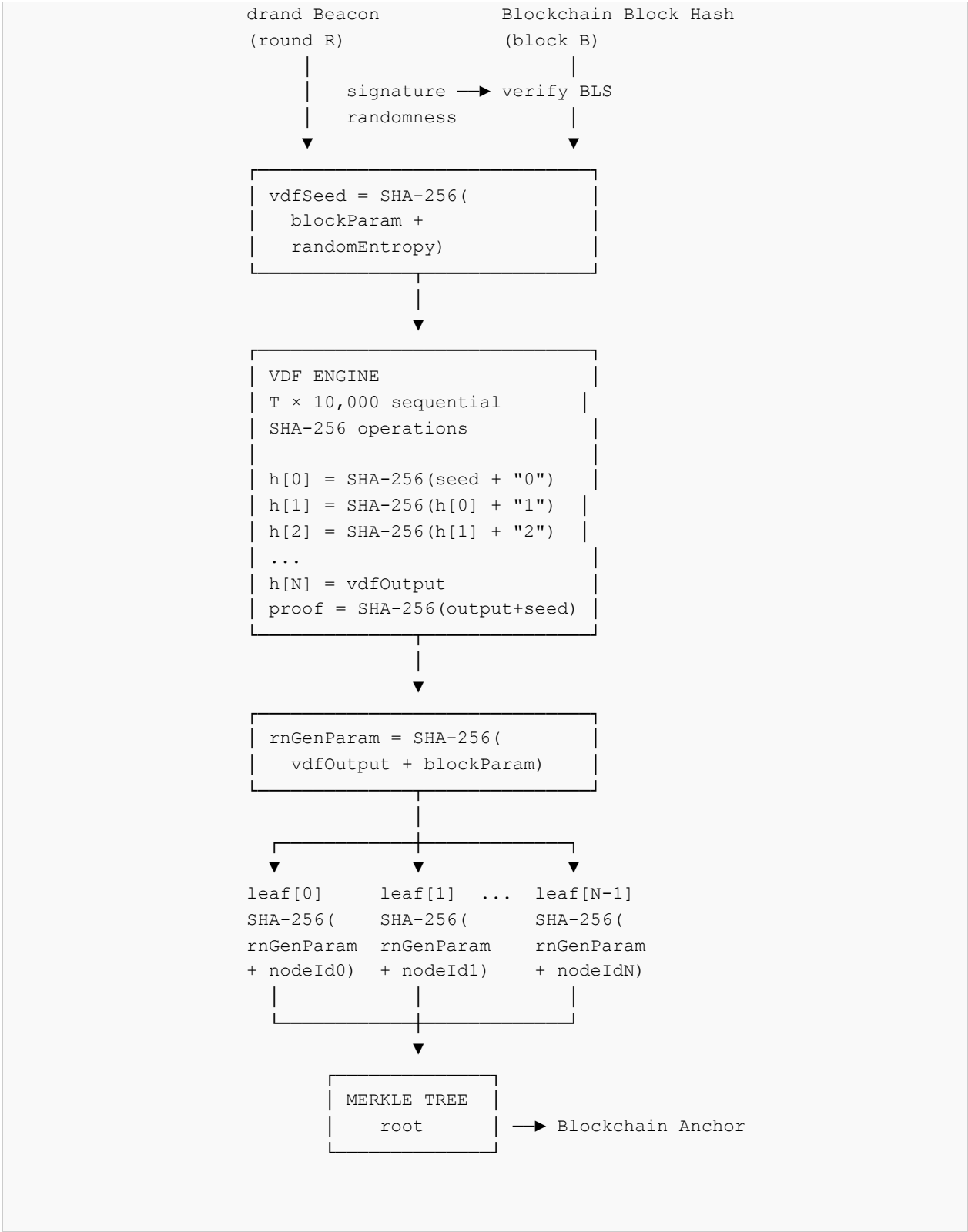


COMPLETE TEMPORAL INTEGRITY:

Before $t=T$: Values UNKNOWABLE (VDF guarantee)
After $t=T$: Values UNALTERABLE (Merkle guarantee)
After $t=T+\Delta$: Values IMMUTABLY TIMESTAMPED (blockchain guarantee)

FIG. 3 — Batch derivation pipeline showing the transformation from VDF output through rnGenParam derivation, leaf hash generation, and Merkle tree construction.

FIG. 3 — Batch Derivation Pipeline



CLAIMS

What is claimed is:

1. A computer-implemented method for generating a batch of random numbers with anti-preview guarantees, comprising:

- (a) obtaining an entropy value from at least one independent entropy source;
- (b) constructing a VDF input seed by combining the entropy value with at least one additional entropy contribution using a cryptographic hash function;
- (c) computing a VDF output by processing the VDF input seed through a Verifiable Delay Function requiring a minimum sequential computation time T , wherein the VDF output is not determinable until said sequential computation time T has elapsed;
- (d) deriving a random number generation parameter from the VDF output;
- (e) deterministically generating a plurality of random values from the random number generation parameter, wherein each random value is derived as a cryptographic hash of the random number generation parameter concatenated with a unique identifier;
- (f) organizing the plurality of random values as leaves of a Merkle tree and computing a Merkle root hash; and
- (g) anchoring the Merkle root hash to an immutable ledger,

wherein the VDF computation time T provides an anti-preview guarantee that no party, including the system operator, can know any of the plurality of random values before the VDF computation completes.

2. The method of claim 1, wherein the at least one independent entropy source comprises a distributed randomness beacon that produces publicly verifiable random values using BLS threshold signatures, and wherein the entropy value is verified by confirming that the cryptographic hash of the beacon's signature equals the beacon's published randomness value.
3. The method of claim 2, further comprising a fallback mechanism wherein, if the distributed randomness beacon is unavailable, the entropy value is obtained from a local entropy source comprising operating system cryptographic randomness combined with at least one blockchain block hash.
4. The method of claim 1, wherein the Verifiable Delay Function comprises an iterative sequential hash chain in which each iteration computes a cryptographic hash of the previous iteration's output concatenated with an iteration counter, thereby preventing parallelization of the computation.
5. The method of claim 1, wherein step (g) of anchoring the Merkle root hash is performed asynchronously after the plurality of random values are made available for consumption, and wherein each random value is tagged with an anchor status that is updated upon confirmation of the anchoring transaction.
6. The method of claim 1, wherein the plurality of random values comprises at least 65,536 values generated from a single VDF output, and wherein each individual value is independently verifiable against the Merkle root hash using a proof path of $O(\log N)$ hash comparisons.
7. The method of claim 1, further comprising injecting the plurality of random values into a pre-generated token pool for serving to consumers at sub-2-millisecond latency, wherein

each served token includes the random value, a Merkle proof path, the batch identifier, and the VDF provenance metadata.

8. A system for generating verifiably unpredictable batch random numbers, comprising:

- (a) an entropy acquisition module configured to obtain entropy values from at least one distributed randomness beacon and at least one blockchain;
- (b) a Verifiable Delay Function engine configured to compute a VDF output from a seed derived from the entropy values, requiring a minimum sequential computation time T ;
- (c) a batch derivation module configured to deterministically generate N random values from the VDF output, wherein N is at least 1,024;
- (d) a Merkle tree builder configured to organize the N random values as leaves and compute a root hash;
- (e) a blockchain anchoring module configured to record the root hash on an immutable ledger; and
- (f) a token pool configured to store and serve the random values at sub-millisecond latency,

wherein the VDF engine provides a temporal barrier ensuring that the random values are not determinable before the VDF computation completes.

9. The system of claim 8, further comprising a plurality of VDF worker threads, each capable of computing a VDF for a separate batch concurrently, and a priority-based refill queue that schedules batch generation based on pool depletion urgency.

10. The system of claim 8, further comprising a shared token pool pre-warmed with a configurable number of VDF-protected batches, wherein multiple consumers consume

tokens from the shared pool with per-consumer rate limiting, and wherein new consumers access tokens immediately without per-consumer batch generation delay.

11. The system of claim 8, wherein the blockchain anchoring module operates asynchronously, injecting tokens into the token pool immediately upon batch generation while anchoring proceeds in background, and wherein tokens served before anchoring confirmation are tagged with a pending anchor status that is updated upon transaction confirmation.

12. The system of claim 8, further comprising an entropy source fallback hierarchy in which a distributed beacon is preferred, and local VDF computation with operating system entropy is used when the beacon is unavailable, and wherein the entropy source used for each batch is recorded in the batch metadata for audit purposes.

13. A computer-implemented method for providing dual temporal integrity guarantees for batch random number generation, comprising:

- (a) generating a batch of N random values using a Verifiable Delay Function pipeline, wherein the VDF computation time T guarantees that the values could not have been known before time T elapsed (anti-preview guarantee);
- (b) organizing the N random values into a Merkle tree whose root hash guarantees that the values cannot be altered after commitment (anti-tamper guarantee); and
- (c) anchoring the Merkle root hash to an immutable blockchain ledger, providing a timestamped commitment that the values existed at or before the anchoring time (temporal proof),

whereby the combination of the VDF anti-preview guarantee, the Merkle anti-tamper guarantee, and the blockchain temporal proof provides complete temporal

integrity: the values were neither knowable before generation nor alterable after generation.

14. The method of claim 13, wherein the batch of N random values is served to consumers from a pre-generated pool at sub-2-millisecond latency, and wherein each served token includes provenance metadata identifying the entropy source, VDF computation parameters, Merkle proof path, and blockchain anchor reference.

15. The method of claim 13, wherein the VDF pipeline incorporates entropy from a distributed randomness beacon whose output is independently verifiable by any third party using the beacon's published public key, thereby providing a publicly auditable entropy source that the system operator cannot control or predict.

16. A non-transitory computer-readable medium storing instructions that, when executed by one or more processors, cause the one or more processors to:

- (a) obtain entropy from a distributed randomness beacon;
- (b) derive a VDF input seed from the obtained entropy and at least one blockchain block hash;
- (c) compute a VDF output by performing a sequential computation requiring at least T seconds;
- (d) derive a batch of random values deterministically from the VDF output;
- (e) construct a Merkle tree over the batch of random values;
- (f) inject the batch into a token serving pool;
- (g) anchor the Merkle root to a blockchain; and
- (h) serve individual tokens from the pool with Merkle proofs and VDF provenance metadata.

ABSTRACT

A method and system for generating batches of random numbers with verifiable anti-preview guarantees using Verifiable Delay Functions (VDFs). The system obtains entropy from independent sources including distributed randomness beacons and blockchain block hashes, processes the entropy through a VDF requiring a minimum sequential computation time T , and deterministically derives a batch of N random values from the VDF output. The batch is organized as a Merkle tree whose root hash is anchored to a blockchain. The VDF computation time provides an anti-preview guarantee: no party, including the system operator, can know the batch values before the VDF computation completes. The Merkle tree provides an anti-tamper guarantee: no value can be altered after the root is committed. Together, these guarantees provide complete temporal integrity — values are neither knowable before generation nor alterable after generation. The system supports sub-2-millisecond token serving from pre-generated pools, multi-worker VDF computation for parallel batch generation, shared and dedicated pool architectures, and hierarchical entropy source fallback.